

Propeller 2 • Application Note P2AN004

Read Real-World Sensors by Frequency, Rotation, and RC Timing on a P2 Pin

Three smart-pin instruments that turn a transducer into a number — no external counter or ADC

November 2026

Version 1.0.4

What You'll Build

Read a resistive, capacitive, light, or rotary sensor directly on a P2 pin — the smart pin does the measuring, the cog just reads the answer.

One shared idea — let a smart pin time or count the signal so the cog never has to — then a small catalog of sensor recipes you choose among by what you are reading:

- **R1 — RC-decay reader** — read a photocell, thermistor, or pot by timing a capacitor's discharge
- **R2 — light-to-frequency reader** — turn a TSL235R's frequency into an irradiance value
- **R3 — quadrature-knob instrument** — a drop-in encoder knob with range clamp, preset, and a debounced button

Applies to: P2 (Propeller 2) silicon • Spin2 / PNut, current release • P2 Edge or P2 Eval board; R3 self-tests with two jumper wires.

Iron Sheep Productions, LLC

P2 Knowledge Base Project

Abstract

A P2 smart pin can time a pulse, count cycles, or decode a rotation directly on the pin, with no cog cycles spent watching it. That is exactly what reading a sensor comes down to — *how long*, *how often*, *how many*. This note turns that capability into three reusable instruments: an RC-decay reader for resistive and capacitive sensors, a light-to-frequency reader, and a production quadrature-encoder knob. Each rests on a smart-pin measurement mode the **P2 I/O & Smart Pins User Guide** documents in full; this note shows how to *apply* those modes to real transducers.

What you'll build: three short, runnable programs — a photocell/thermistor reader, a TSL235R light-to-frequency reader, and a drop-in encoder knob — the last of which verifies itself on the bench with two jumper wires and no encoder at all.

Prerequisites & Hardware

You should already be comfortable with:

- Configuring a smart pin (WRPIN / WXPIN / WYPIN, or the Spin2 `pinstart` that wraps them) and reading it back with RDPIN.
- Reading and writing Spin2.
- The P2's smart-pin measurement modes at the mechanism level. This note *uses* those modes; it does not re-teach them. For how each mode works — its registers, its modifiers, its worked examples — see the **P2 I/O & Smart Pins User Guide** (a companion P2 Knowledge Base publication): time measurement in Chapter 13, counting and quadrature in Chapter 14, frequency and period in Chapter 15.

Hardware:

- A P2 Edge or P2 Eval board.
- **For R3, two jumper wires** — the encoder recipe drives its own A/B signals from a second pin pair, so you can confirm it end to end with no encoder attached.
- **For R1 and R2, the sensor itself** — a photocell or thermistor plus a small capacitor and a series resistor for R1; a TSL235R light-to-frequency sensor for R2. These recipes compile and run without the part, but their *absolute readings* mean nothing until the transducer is wired and calibrated.

The Idea

Reading a sensor is almost always a measurement of **time** or **count**, and the P2 smart pin makes exactly those measurements in hardware:

- A **resistive or capacitive** sensor changes *how long* a capacitor takes to discharge — a **time**.
- A **light-to-frequency** sensor changes *how often* its output toggles — a **frequency**, which is a count of periods over a known time.

- A **rotary encoder** changes *how many* quadrature edges go by, and in which direction — a signed **count**.

In every case the same pattern holds: you configure a smart pin for the right measurement mode, point it at the pin, let it run, and read a number. The cog never has to poll the signal edge by edge — the smart pin already did. The three recipes in this note differ only in *which* measurement and *which* transducer; the shape — **configure the mode, let the smart pin run, read the result** — is one idea, applied three ways.

How It Works (in brief)

The measurement modes themselves live in the **P2 I/O & Smart Pins User Guide**; this is just the working summary of the four this note uses — three recipes, but R2 runs two modes together — with a pointer to where each is documented in full.

Mode	Constant	Measures	Documented in
%10001	P_HIGH_TICKS	clocks an input stays high (one decay time)	Guide Ch. 13.3
%10101	P_COUNTER_TICKS	actual elapsed clocks of a period-aligned window	Guide Ch. 15.3
%10111	P_COUNTER_PERIODS	whole signal periods within that window	Guide Ch. 15.3
%01011	P_QUADRATURE	signed A/B transition count (4 per detent)	Guide Ch. 14.2

Two of these work together. The **reciprocal frequency counter** (R2) runs `P_COUNTER_PERIODS` and `P_COUNTER_TICKS` over the *same* window: one returns the number of whole periods, the other the exact clock count that window actually took. Frequency then falls out of a single divide:

frequency = periods × clkfreq ÷ ticks

This is more accurate at low frequencies than counting edges in a fixed gate, because the window stretches to end on a whole period — no partial cycle is ever miscounted. For example, a 50 kHz signal measured over a 100 ms gate at 200 MHz gives about **5,000 periods** in about **20,000,000 ticks**, so `MULDIV64(5_000, 200_000_000, 20_000_000)` is **50,000 Hz** exactly. (See the Guide, Ch. 15.3, for the full reciprocal-counter treatment.)

`P_HIGH_TICKS` (R1) and `P_QUADRATURE` (R3) each return a single value you read with RDPIN — a high-time in clocks, and a signed position count, respectively.

Choosing a Recipe

Pick by the transducer you are reading:

If you're reading…	Use	The mode(s)
a resistance or capacitance — photocell, thermistor, pot	**R1** — RC-decay reader	%10001 P_HIGH_TICKS
a light- or voltage-to-frequency sensor	**R2** — light-to-frequency reader	%10111 + %10101 reciprocal counter
a rotary / quadrature encoder knob	**R3** — quadrature-knob instrument	%01011 P_QUADRATURE

Each recipe is a complete, runnable program; each ends with a ♦ **Verify** that tells you how to confirm it on your own bench.

Recipe R1 — RC-Decay Reader

Use it when you are reading any sensor whose **resistance or capacitance** varies — a CdS photocell, a thermistor, a potentiometer, a soft-pot.

The trick is the classic "rctime" measurement, done in hardware. Wire the pin through a small series resistor to a node that has a capacitor to ground and the sensor (as a resistance) to ground:

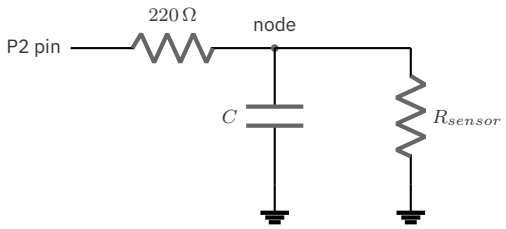


Figure 1. R1 — RC-decay network: the pin charges the node through the series resistor, then floats so the capacitor discharges through the sensor resistance.

Drive the pin high and the capacitor charges. Then **float** the pin and the capacitor discharges through the sensor resistance. The smart pin, in `P_HIGH_TICKS` mode, times how long the node stays above the input's Schmitt threshold — and that discharge time is proportional to R·C. A brighter light lowers a CdS cell's resistance, the node falls faster, the count drops; the clock count *is* the sensor reading.

The pin configuration carries the whole trick. `P_SCHMITT_A` gives a clean threshold crossing on the slow RC ramp. `P_OE | P_LOW_FLOAT` makes the pin **drive when its output is high** (to charge) but **float when its output is low** (so the cap discharges through the sensor, not through the pin's driver). Each half is doing a named job: `P_LOW_FLOAT` selects *float* as the low-side drive strength, leaving the high side at its default fast drive, and `P_OE` is documented as *"enable output in smart*

pin mode, regardless of DIR" — which is what lets the smart pin work the output here without the DIR handling a plain drive-strength configuration would need:

```
CON
  _clkfreq = 200_000_000

  RC_PIN  = 8                  ' R-C sensor node (220 ohm series + cap)
  CHG_MS  = 5                  ' capacitor charge time, milliseconds

  ' Time the high duration of the decaying node: %10001 P_HIGH_TICKS.
  ' P_SCHMITT_A gives a clean threshold crossing on the RC ramp.
  ' P_OE | P_LOW_FLOAT lets the pin DRIVE high to charge the cap, then
  ' FLOAT (high output drives, low output floats) so it discharges
  ' through the sensor resistance while the smart pin times the high.
  RC_MODE = P_HIGH_TICKS | P_SCHMITT_A | P_OE | P_LOW_FLOAT

PUB main() | clocks, us
  WRPIN(RC_PIN, RC_MODE)             ' configure the timing smart pin once
  repeat
    clocks := read_rc()              ' one charge-float-time cycle
    us     := clocks / (clkfreq / 1_000_000)
    DEBUG(udec(clocks), " clocks  (", udec(us), " us)")
    waitms(200)

PRI read_rc() : clocks
  ' 1) charge: drive the node high through the series resistor
  pinh(RC_PIN)                       ' OUT=1 -> P_LOW_FLOAT drives high
  waitms(CHG_MS)
  ' 2) release: reset the smart pin, then float and start timing
  pinf(RC_PIN)                       ' DIR low resets the accumulator
  pinl(RC_PIN)                       ' OUT=0 -> P_LOW_FLOAT floats; DIR
  '                                    high re-enables, timing starts now
  ' 3) wait for the high->low crossing, read the latched high-tick count
  repeat until pinread(RC_PIN)       ' IN raised when the node falls
  clocks := RDPIN(RC_PIN)
```

rc-decay-reader.spin2

How this works. WRPIN sets the mode once. Each `read_rc` runs one cycle: `pinh` drives the pin high — because of `P_LOW_FLOAT`, a high output really does drive, so the capacitor charges through the 220 Ω resistor. After the charge delay, `pinf` lowers DIR (which resets the smart pin's accumulator) and `pinl` raises it again with the output low — `P_LOW_FLOAT` floats the pin, the capacitor begins discharging, and the freshly-reset smart pin starts timing the still-high node. When the node finally falls through the Schmitt threshold, the smart pin latches the high-tick count and raises IN; the polling loop reads it with RDPIN.

■ **Hardware:** The 220 Ω series resistor sets a known, current-limited charge path and protects the pin. The capacitor value sets the timescale — too small and the discharge is over before the smart pin sees it; too large and each reading is slow. Size C so a mid-range sensor resistance gives a comfortable millisecond-scale count.

▲ **Pitfall — get the float direction right.** The measurement depends on `P_LOW_FLOAT`: when the output goes low the pin must *float* so the capacitor discharges through the sensor. Without it, a low output actively drives the node to ground and you time the pin's driver, not your sensor — the count collapses to near zero every time.

♦ **Verify:** R1's *absolute* count depends on your exact R, C, and the pin's Schmitt thresholds, so it is calibrated against the sensor, not predicted from a formula. With a CdS photocell wired, the relationship is what you confirm first: shade the cell and the count climbs (higher resistance, slower discharge); brighten it and the count drops. Two symptoms tell you the wiring is wrong. If the read **hangs** (no new line prints), the node never fell back through the threshold, so the high→low crossing that latches the count never happens — either it never charged high (charge time too short, or the series path open), or it charged but has no discharge path (the sensor is disconnected). A count **near zero** means the node was pulled low almost immediately after timing began — classically a missing `P_LOW_FLOAT`, so a low output actively drives the node to ground instead of floating and the high time collapses to nothing. Absolute accuracy — turning the count into lux or degrees — comes from a two-point calibration against a reference, and is confirmed on hardware in a later pass. (If nothing prints at all, you built without DEBUG enabled — compile with `pnut-ts -d`.)

Recipe R2 — Light-to-Frequency Reader

Use it when you are reading a **TSL235R** — or any sensor that reports its value as an output frequency.

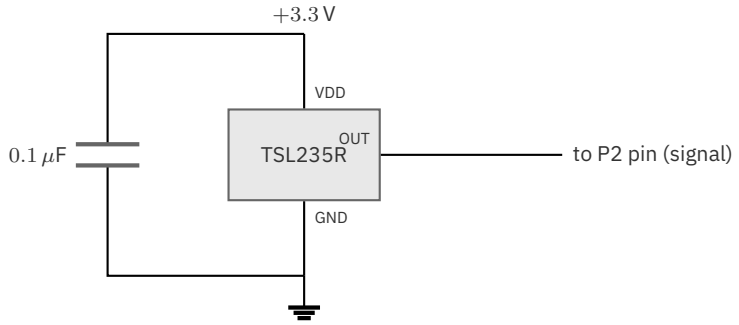


Figure 2. R2 — TSL235R hookup: VDD to 3.3 V, GND to ground, OUT to one P2 pin, with a 0.1 μF decoupling capacitor close to the device.

The TSL235R emits a square wave whose frequency is proportional to the light falling on it. Reading it means measuring that frequency and converting it to an irradiance. The measurement is the reciprocal frequency counter from *How It Works*: two consecutive smart pins over one window — one counts whole periods, the other records the exact elapsed clocks — and `frequency = MULDIV64(periods, clkfreq, ticks)`. The signal is physically present on `SIG_PIN`; the periods counter on `SIG_PIN+1` routes its inputs back one pin (`P_MINUS1_A | P_MINUS1_B`) so both pins watch the same signal:

```
CON
  _clkfreq = 200_000_000

  SIG_PIN  = 16               ' TSL235R OUT -> this pin (the signal)
  '          17              ' periods counter, routed from SIG_PIN
  GATE_MS  = 100              ' minimum gate window, milliseconds

  ' Reciprocal frequency counter, two consecutive smart pins, one window:
  '   SIG_PIN   : %10101 P_COUNTER_TICKS   -> elapsed clocks (A/B local)
  '   SIG_PIN+1 : %10111 P_COUNTER_PERIODS -> complete periods, with its
  '               A/B routed back one pin (P_MINUS1_A|P_MINUS1_B) so it
  '               watches the SAME signal physically present on SIG_PIN.
  '   freq = MULDIV64(periods, clkfreq, ticks)
  TIK_MODE = P_COUNTER_TICKS
  PER_MODE = P_COUNTER_PERIODS | P_MINUS1_A | P_MINUS1_B

  ' TSL235R datasheet typical: fO = 250 kHz at Ee = 430 uW/cm^2
  ' (lambda_p = 635 nm, VDD = 5 V). Irradiance scales with frequency:
  '   Ee[uW/cm^2] = freq[Hz] * 430 / 250_000
  CAL_UW   = 430
  CAL_HZ   = 250_000

PUB main() | gate, freq, ee_x100, ee_int, ee_frac
  gate := (clkfreq / 1000) * GATE_MS
  pinstart(SIG_PIN+0, TIK_MODE, gate, %00)    ' elapsed-ticks pin
  pinstart(SIG_PIN+1, PER_MODE, gate, %00)    ' periods pin
  repeat
    freq    := read_freq()
    ' irradiance in hundredths of a uW/cm^2 (integer fixed point)
    ee_x100 := muldiv64(freq, CAL_UW * 100, CAL_HZ)
    ee_int  := ee_x100 / 100
    ee_frac := ee_x100 // 100
    DEBUG("TSL235R  freq = ", udec_(freq), " Hz", 13)
    DEBUG("  Ee = ", udec_(ee_int), ".", udec_(ee_frac), " uW/cm2")
    waitms(300)

PRI read_freq() : freq | periods, ticks
  ' block until both pins have closed their period-aligned window
  repeat until pinread(SIG_PIN+0) AND pinread(SIG_PIN+1)
  ticks   := RDPIN(SIG_PIN+0)
  periods := RDPIN(SIG_PIN+1)
  if ticks                            ' no signal -> guard divide-by-zero
    freq := muldiv64(periods, clkfreq, ticks)
```

light-to-freq-reader.spin2

How this works. Both pins start together on a `GATE_MS` window, so their measurements span the same period-aligned interval. `read_freq` waits until both have raised IN, reads the elapsed `ticks` and the `periods` count, and computes the frequency with MULDIV64 so the `periods * clkfreq` product never overflows 32 bits. The conversion to irradiance comes straight from the TSL235R datasheet's typical operating point — 250 kHz at 430 µW/cm² — so `Ee = freq * 430 / 250_000`; the code carries two decimal places by scaling by 100.

■ **Hardware:** The TSL235R runs on 2.7–5.5 V, so it works directly at the P2's 3.3 V. The datasheet's 250 kHz / 430 µW/cm² figure is specified at 5 V, so the built-in conversion is a starting point — for absolute irradiance, calibrate against a reference light source at your actual supply voltage. A 0.1 µF capacitor close to the sensor between supply and ground keeps its output clean.

▲ **Pitfall — dark is not zero.** The TSL235R's dark output frequency is a few hertz (the datasheet lists 0.4–10 Hz), not zero. Treat a very low reading as "near dark," not as a fault, and don't divide by a frequency you haven't confirmed is real.

★ **Tip:** Choose `GATE_MS` for your light range. A longer window resolves low light more finely (more periods to count); a shorter window updates faster in bright light where there are periods to spare.

♦ **Verify:** With the sensor lit, the frequency rises as you brighten the light and falls as you shade it; cover it fully and the reading drops toward the few-hertz dark frequency. If the frequency reads zero no matter what, the signal isn't reaching `SIG_PIN` (check the wiring) or `ticks` came back zero (no edges in the window). Erratic, jumpy numbers usually mean the sensor isn't driving the pin or the supply is noisy: check the wiring and the supply capacitor. If the line can be left disconnected, an external pull-down gives it a defined level. The mapping from frequency to absolute irradiance is confirmed against a calibrated source on hardware in a later pass; the frequency measurement itself is exact.

Recipe R3 — Quadrature-Knob Instrument

Use it when you want a rotary encoder as a **user-interface knob** — menu navigation, value entry, a volume control.

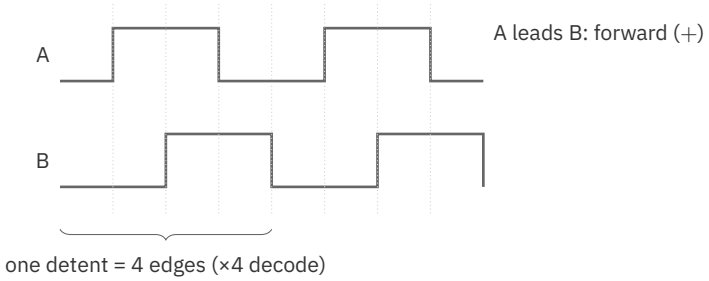


Figure 3. R3 — quadrature A/B timing: the two channels are a quarter-cycle apart, so each detent produces four counted edges; which channel leads sets the sign.

`P_QUADRATURE` decodes the encoder's A and B signals into a signed count, four counts per detent, and the **P2 I/O & Smart Pins User Guide** Ch. 14.2 documents that mode in full. What a UI actually needs, though, is more than the raw count: it wants the count normalized to detents, an adjustable zero, a clamp to a usable range, and a debounced button. This recipe wraps the mode into that drop-in instrument — and it stimulates itself, so you can verify it with two jumpers and no encoder:

```
CON
  _clkfreq = 200_000_000

  ENC_A  = 32                  ' encoder A input (smart pin)
  ENC_B  = 33                  ' encoder B input (B = A+1, P_PLUS1_B)
  BTN    = 34                  ' pushbutton input (active low; -1 if none)

  STIM_A = 40                  ' Tier-0 self-test: drive A (jumper 40->32)
  STIM_B = 41                  '                   drive B (jumper 41->33)

  DET4X  = TRUE                ' detented encoder: 4 raw counts per detent
  PRESET = 0                   ' initial detent value
  LOLIM  = 0                   ' clamp range, low
  HILIM  = 20                  ' clamp range, high

VAR
  LONG mod4x                   ' 0 or 2: the 4x-per-detent normalize shift
  LONG offset                  ' preset/zero bookkeeping

PUB main() | pos
  enc_start(DET4X, PRESET)
  ' --- Tier-0 self-test (jumper 40->32 and 41->33; no encoder) ---
  stim_setup()
  stim_turn(5, TRUE)                    ' 5 detents forward
  DEBUG("Tier-0: 5 detents fwd  -> reads ", sdec_(enc_value()))
  stim_turn(2, FALSE)                   ' 2 detents back
  DEBUG("Tier-0: 2 detents back -> reads ", sdec_(enc_value()))
  ' --- live read loop (turn a real encoder knob) ---
  repeat
    pos := enc_value()
    DEBUG("position = ", sdec_(pos), "   button = ", udec_(enc_button(25)))
    waitms(200)

PUB enc_start(d4x, pval)
  ' One smart pin in %01011 P_QUADRATURE; B is the next pin up.
  ' Continuous totalizer (X=0): Z always holds the signed 4x count.
  pinstart(ENC_A, P_QUADRATURE | P_PLUS1_B, 0, 0)
  mod4x := (d4x) ? 2 : 0               ' 4 counts/detent -> sar 2
  enc_set(pval)

PUB enc_set(pval) : result
  ' Zero/preset without dropping the smart pin: pulse DIR, set offset.
  pinf(ENC_A)                          ' DIR low resets the count
  pinl(ENC_A)                          ' DIR high re-enables at zero
  offset := pval
  return pval

PUB enc_raw() : result
  ' Un-offset detent count: raw signed 4x count normalized by mod4x.
  return RDPIN(ENC_A) SAR mod4x

PUB enc_value() : result
  ' Detent value with preset applied, clamped to LOLIM..HILIM.
  result := enc_raw() + offset
```

continues on next page →

↪ continued from previous page

```
  if result < LOLIM
    result := enc_set(LOLIM)
  elseif result > HILIM
    result := enc_set(HILIM)

PUB enc_button(ms) : pressed
  ' Debounced active-low button: 1 only if held low for ms milliseconds.
  if BTN < 0
    return 0
  if pinread(BTN) == 0
    pressed := 1
    repeat (ms #> 0)
      waitms(1)
      pressed &= pinread(BTN) ^ 1

PUB stim_setup()
  ' Tier-0 stimulus pins idle low, then re-zero the encoder.
  pinl(STIM_A)
  pinl(STIM_B)
  waitms(1)
  enc_set(PRESET)

PUB stim_turn(detents, fwd)
  ' Walk the A/B gray-code sequence: 4 transitions = 1 detent.
  ' fwd uses one phase order; reverse swaps it (sign flips).
  repeat detents
    if fwd
      stim_ab(0, 1)
      stim_ab(1, 1)
      stim_ab(1, 0)
      stim_ab(0, 0)
    else
      stim_ab(1, 0)
      stim_ab(1, 1)
      stim_ab(0, 1)
      stim_ab(0, 0)

PRI stim_ab(a, b)
  pinw(STIM_A, a)
  pinw(STIM_B, b)
  waitus(50)                            ' let the decoder register the edge
```

quadrature-knob.spin2

How this works. `enc_start` configures one smart pin in `P_QUADRATURE` with `P_PLUS1_B`, which takes the B signal from the next pin up (so B must sit within a few pins of A). In continuous mode (X = 0) the pin's count register always holds the live signed total. `enc_raw` reads that count and shifts it right by `mod4x` (2 for a detented encoder) to normalize four-counts-per-click down to one-per-click. `enc_value` adds the `offset` and clamps the result to `LOLIM..HILIM`; `enc_set` re-zeros the hardware count and records a new offset by pulsing DIR. `enc_button` debounces an active-low button by requiring it to read low for a full window of milliseconds. The `stim_*` routines drive a second pin

pair through the A/B gray-code sequence — four transitions per detent — so the instrument can exercise itself.

★ **Tip:** `mod4x` is the detent knob. A detented encoder clicks four counts per detent, so `SAR 2` gives one-per-click; an encoder *without* detents wants `mod4x := 0` to keep every count.

▲ **Pitfall — B must be near A.** `P_PLUS1_B` reads B from `ENC_A + 1`. If your encoder's B line is on a pin more than a few away from A, the routing constant won't reach it — keep A and B on adjacent pins, or pick the routing modifier that matches your wiring (see Ch. 14.2).

■ **Hardware:** Mechanical encoders bounce. If a turned knob jumps or counts backward, add `P_FILT1_AB` to the mode to route the A/B inputs through the global FILT1 digital filter (reset default ≈600 ns low-pass) — it costs nothing and cleans up most contact noise.

♦ **Verify:** This is the recipe you can fully confirm with no encoder. Jumper pin 40 to pin 32 and pin 41 to pin 33, and run: the program drives five detents forward and prints the reading, then two back. The expected magnitudes are the known answers — **5**, then **3** — because the stimulus generates exactly that many quadrature cycles. The *sign* depends on your phase order and wiring: if "forward" reads −5, your A/B pair is reversed; swap the two jumpers (or the pin order) to flip it. If the reading stays 0, a jumper is missing or on the wrong pin; if it reads four times the expected magnitude, `mod4x` is 0 (detent normalization off); if it is erratic, add `P_FILT1_AB`.

Going Further / Adapt

Each recipe generalizes by changing only its constants. R1 reads any resistive or capacitive sensor — swap the photocell for a thermistor or a pot, adjust `CHG_MS` and the capacitor for the new timescale, and recalibrate. R2 reads any frequency-output sensor — a voltage-to-frequency converter, a different light-to-frequency part — by changing only the `CAL_UW` / `CAL_HZ` conversion constants. R3 becomes a **velocity** readout by running a second quadrature pin in periodic mode (X > 0), which the Guide's Ch. 14.2 works through.

The background-ISR ceiling. R1 ships the *polled* form — the cog calls `read_rc` when it wants a reading. The original P2_rctime object (OBEX #2831) goes further: it loads a small PASM2 interrupt service routine via REGEXEC into high cog RAM (`ORG $1B0`) that services each measurement in the *background* of the Spin2 interpreter (returning with RETI1), continuously updating a hub variable. The cog then never polls at all — it just reads the latest value whenever it likes. That terminate-stay-resident pattern is the next step when you want a reading maintained automatically with zero foreground cost; the complete implementation is in OBEX #2831.

Where the worked instruments live. This note is narrowed to *sensor* recipes — reading a transducer. The classic worked *instruments* built on these same modes — a bench **frequency counter**, an **RPM / tachometer**, an **ultrasonic time-of-flight** ranger — are worked end to end in the **P2 I/O & Smart Pins User Guide** (Chapters 13, 14, and 15). Reach for the Guide when you need one of those; reach for this note when you have a transducer to read.

See It Work / Verify

The three recipes verify at different levels, and it is worth being clear about which is which.

R3 and the measurement layer are self-verifiable. The encoder recipe's two-jumper self-test reads a known detent count with no external hardware — drive five detents, read five. That confirms the smart-pin configuration, the count normalization, the offset, and the clamp, all on a bare Edge board.

R1 and R2 are confirmed for *mechanism* now, and for *accuracy* on hardware. Both compile and run, and both show the right *behavior* — R1's count moves with sensor resistance, R2's frequency moves with light. Turning those into a calibrated lux, a temperature, or an absolute irradiance requires a known reference (a calibrated R-C network, a reference light source). Those calibration runs are pending; when accepted they are recorded in the project's empirical-findings ledger. This note does not print invented sensor numbers — the values you see are the ones the hardware produces.

The one failure mode common to all three: nothing in the DEBUG window means you built without DEBUG enabled. Compile with `pnut-ts -d`.

Pitfalls & Notes

▲ **Pitfall — `P_LOW_FLOAT` is what makes R1 work.** The RC-decay read depends on the pin floating when its output is low, so the capacitor discharges through the sensor rather than through the pin's driver. Drop `P_LOW_FLOAT` and every reading collapses toward zero.

★ **Tip — reciprocal beats fixed-gate at low frequency.** R2 counts *whole periods* and the *exact* time they took, rather than counting edges in a rigid gate. At low frequencies, where a fixed gate might catch only a handful of edges and miscount the partial cycle at each end, the reciprocal method stays precise. (Guide, Ch. 15.3.) Because it averages over the whole window, though, it reports a *mean* frequency and will not reveal cycle-to-cycle jitter — reach for a different technique when short-term stability, not average rate, is what you care about.

★ **Tip — keep the measured frequency well below the system clock.** The smart pin samples the input against `clkfreq`, so a clean reciprocal reading needs many system-clock ticks per input period. Stay below roughly half the system clock, with headroom — at 200 MHz that leaves enormous room for the audio- and sensor-rate signals these recipes target; the ceiling only bites for RF-rate inputs.

■ **Hardware — a reading is only as accurate as the clock.** Every frequency here is computed against `clkfreq`, so the measurement inherits the accuracy of the P2 system clock. A crystal's tolerance and loading pull the clock a few ppm, which sets the floor on absolute accuracy; for parts-per-million work, discipline the clock against a known reference before trusting the last digits.

■ **Hardware — stay at a legal clock.** These programs run at 200 MHz — the top of the P2 PLL VCO's designed 100–200 MHz range, so they sit comfortably within spec. There is no reason to

overclock for any of these measurements — resolution comes from integration time and from the reciprocal method, not from a faster clock.

♦ **Verify — the self-test's sign is wiring-dependent, its magnitude is not.** R3's self-test always produces the right *number* of detents; whether "forward" reads positive or negative depends on which stimulus pin lands on A and which on B. Use the magnitude as the known answer and the sign to check your A/B orientation.

Conclusion

One idea runs under all three recipes: let the smart pin time or count the signal, and the cog only has to read the answer. A resistance becomes a discharge time, a light level becomes a frequency, a turned knob becomes a signed count — and in every case the hardware has already done the measuring before the cog looks. Start from the recipe that matches your transducer; confirm the encoder path with the two-jumper self-test, which needs no parts at all; and calibrate the analog recipes against a known reference before you trust an absolute number. The modes themselves — and the bench instruments built on them — are in the **P2 I/O & Smart Pins User Guide**; this note is the bridge from those modes to a real sensor on a pin.

Resources

- **Example code (download).** The three recipes in this note are published together as the **P2AN004 source ZIP** beside this note's PDF — `rc-decay-reader.spin2`, `light-to-freq-reader.spin2`, and `quadrature-knob.spin2`, each a complete `pnut-ts`-compiled program ready to build and run.
- **P2 I/O & Smart Pins User Guide, Chapters 13–15** (a companion P2 Knowledge Base publication) — the measurement modes this note applies (time, counting/quadrature, frequency/period), and the worked frequency-counter, tachometer, and time-of-flight instruments this note points to.
- **OBEX #2831 — P2_rctime** (phonoclese): the RC-decay donor. Its background-ISR pattern, built on REGEXEC, is the one described in *Going Further*. Find it by object number at the Parallax OBEX (`obex.parallax.com`).
- **OBEX #2829 — Quadrature Encoder** (Jon "JonnyMac" McPhalen): the production-encoder donor behind R3 — range limiting, preset, det4x, and the debounced button.
- **TSL235R light-to-frequency sensor** (Parallax Quick Bytes): the sensor and demo code behind R2.

References

These are the authoritative primary sources the note's facts trace to. (The companion *P2 I/O & Smart Pins User Guide* named in Resources derives from these same sources.)

1. *Propeller 2 Documentation v35 — Rev B/C Silicon* (Chip Gracey, Parallax Inc.) — Smart Pins: the four measurement modes this note uses, each named with its mode number in *How It Works*, and the A/B input routing modifiers.
2. *Spin2 Language Documentation* (Chip Gracey, Parallax Inc.) — `PINSTART`, `WRPIN`, `RDPIN`, `MULDIV64`, and the pin-control methods (`pinh`, `pinl`, `pinf`, `pinw`, and `pinread`).
3. *TSL235R-LF Light-to-Frequency Converter Datasheet* (ams/TAOS) — the output-frequency figure used for the irradiance conversion (output frequency 250 kHz typical at Ee = 430 µW/cm², λp = 635 nm, VDD = 5 V), the 2.7–5.5 V operating range, and the dark output frequency.

Revision History

The example library `P2AN004-src.zip` is versioned with the note, so a ZIP downloaded beside this PDF matches the code printed here. This note's version is on its cover, and the top row below.

Version	Date	Notes
1.0.4	2026-11-06	Recipe R2's ticks pin is plain P_COUNTER_TICKS, and the Verify step's advice on erratic readings checks the wiring and the supply
1.0.3	2026-09-10	The RC-timing pin configuration explained by its parts: P_LOW_FLOAT as the low-side drive strength, and P_OE as *"enable output in smart pin mode, regardless of DIR"*. No instruments added.
1.0.2	2026-08-08	License restored to CC BY-SA 4.0 (share and adapt, including commercially, with attribution and under the same terms); the Trademarks note now states the license grants permissions under copyright only. No technical content changed.
1.0.1	2026-07-11	Specification-accuracy and titling refinement (the note is *Frequency / Rotation / RC-Timing Measurement*, naming its three instruments): the programs run at 200 MHz, the top of the P2 PLL VCO's designed 100–200 MHz range; P_FILT1_AB routes the A/B inputs through the global FILT1 digital filter (reset default ≈600 ns low-pass). No instruments added.
1.0.0	2026-07-03	First community-review release. Techniques-catalog: three smart-pin measurement instruments selected by transducer — an RC-decay reader (P_HIGH_TICKS), a light-to-frequency reader (reciprocal frequency counter, P_COUNTER_TICKS + P_COUNTER_PERIODS), and a quadrature-knob instrument (P_QUADRATURE) with detent normalization, preset, range clamp, and a debounced button. Adds rendered circuit/timing figures. All embedded and example-library code compiles clean under pnut-ts (-d); the encoder self-verifies with a two-jumper known-answer detent count, while the analog recipes' absolute calibration defers to a hardware pass and carries no invented readings. Ships a downloadable example library.

Copyright & License

Copyright © 2026 Iron Sheep Productions, LLC and Parallax Inc.

This work is licensed under the Creative Commons Attribution–ShareAlike 4.0 International License (CC BY-SA 4.0) — you may share and adapt it, including commercially, with attribution and under the same terms. To view the full license, visit https://creativecommons.org/licenses/by-sa/4.0/.

Parallax, Propeller, and Spin are trademarks of Parallax Inc. This license grants permissions under copyright only; it does not grant rights to use these trademarks, and adapted or redistributed copies must not imply endorsement by, or official status with, Iron Sheep Productions, LLC or Parallax Inc.

Acknowledgments

- **Parallax Inc.** — for the Propeller 2 and the reference documentation that forms the foundation of this work.

- **Chip Gracey** — for the design of the P2 smart pins and the measurement modes these recipes apply.
- **The P2 community** — phonoclese (P2_rctime, OBEX #2831) and Jon "JonnyMac" McPhalen (Quadrature Encoder, OBEX #2829), whose published code grounded the RC-decay and encoder recipes; and Francis Bauer, Jon McPhalen, Chip Gracey, and Ken Gracey, whose Parallax Quick Bytes work on the TSL235R grounded the light-to-frequency recipe.